

ARCHITECTURE REVIEW · PART V · SYSTEM MODEL · IMPLEMENTATION PRIORITY · TOOLING

System Model, *Priority* & Implementation Tooling

What the model is · What gets built first · Database schema for states and workflow · Visual graph editor · Storage tooling recommendations for Delphi 12.2 CE

ANALYSIS 01

What the System Model Actually Consists Of

Your identification of state descriptions and workflow as the core elements is correct — but it is worth being precise about what "the model" means in this architecture, because it has three distinct layers that must be kept separate in the implementation.

①	Object Model — the domain objects and their schemas Defines what types of objects exist (PatientRecord, Visit, Provider, Order...), their properties, data types, cardinality, and validation rules. This is the vocabulary of the system. SCHEMA DB
②	Execution Context Contracts — what each API function needs and produces Declares which object properties each command requires and contributes. Maps domain objects to RPC parameters. Built on top of the object model; cannot exist without it. SCHEMA DB
③	Workflow Graph — states, transitions, guard conditions GRAPH DB

	Defines what states (active windows/forms) exist, how they connect, what context each requires and contributes, and what commands are available in each state. References the object model for context property names.	
④	Module Registry — which modules implement which states Maps workflow states to the software modules (forms, frames, console views) that render them. Built last; depends on the workflow graph being stable.	CONFIG
⑤	UI Implementation — the actual forms and screens Renders the current workflow state. The thinnest layer; can be designed and built once the workflow graph is stable.	UI

Critical insight: Layers ①–③ are pure data — schemas, contracts, and graph definitions. They have no code, no UI, no Delphi classes. They can be authored, reviewed, and versioned before a single line of application code is written. The tools to manage these layers are therefore the first things to implement.

ANALYSIS 02

Correct Implementation Priority Order

Your instinct is right. The state descriptions and workflow graph are foundational. Everything else depends on them. But within that, there is a precise dependency order:

S1

Object Model Schema Manager
The tool for defining domain object types, their properties, data types, and validation rules. This is the dictionary everything else references. Must exist first — the workflow graph references object property names; you cannot design transitions until you know what properties exist.

S2

Workflow Graph Editor + Storage
The tool for visually designing and persisting the workflow graph — nodes (states), transitions (edges), transition types, context contracts per node. References the object model from S1 when specifying context property requirements. This is the highest-value deliverable: once complete, clinical workflow review is possible without any UI.

S3

Command Registry + Context Contract Editor

The tool for defining commands, their required/contributed context properties, and their RPC mappings. Depends on S1 (object model) and informs S2 (workflow transitions reference commands). Can be developed in parallel with S2 once S1 is stable.

S4

State Machine Engine + Runtime Loader

The Delphi code that loads the workflow graph from storage (S2) and configures a stateless state machine instance at application startup. No UI; pure engine code. Once this exists, the workflow can be tested and scripted without any form being implemented.

Key observation: S1 and S2 are *authoring tools* — small internal applications whose sole purpose is to manage the model data. They are the first deliverables, not the last. A developer building the patient-selection workflow cannot start until S1 and S2 exist and contain the relevant object definitions and states.

ANALYSIS 03

Storage Strategy — One Database, Two Concerns

Both the object model schema and the workflow graph should be stored in a **single embedded SQLite database** — the system model database. This is the master definition file for the entire application. It is not a runtime data store; it is a *design artifact* that ships with the application and is loaded at startup.

WHY SQLITE, NOT JSON FILES

- **Relational integrity between layers** — the workflow graph references object property names by foreign key, not by string lookup. Changes to the object model are immediately visible as constraint violations in the workflow graph — impossible with flat JSON files.
- **Queryable** — "show all workflow states that require PatientRecord.dfn" is a SQL query, not a recursive JSON scan. This matters for tooling, validation, and impact analysis.

- **Single file, zero deployment** — SQLite is a single .db file. FireDAC supports it natively in Delphi 12.2 CE with static linking; no separate installation required.

- **Version control friendly** — the .db file can be committed to source control. SQLite's file format is stable across versions. Schema migrations are trackable.

- **Transaction safety** — authoring tool edits are ACID transactions. A crash during a complex workflow edit does not corrupt the model.

PROPOSED SCHEMA

System Model Database – Core Tables (SQLite DDL)

```
-- ① OBJECT MODEL
CREATE TABLE ObjectType (
  id          TEXT PRIMARY KEY,          -- e.g. 'PatientRecord'
  label       TEXT NOT NULL,
  description TEXT,
  version     INTEGER DEFAULT 1
);

CREATE TABLE ObjectProperty (
  id          TEXT PRIMARY KEY,          -- e.g. 'PatientRecord.dfn'
  object_type TEXT REFERENCES ObjectType(id),
  name        TEXT NOT NULL,            -- 'dfn'
  data_type   TEXT NOT NULL,            -- 'string', 'integer', 'date', 'ien'
  nullable    INTEGER DEFAULT 1,        -- 0 = always required
  description TEXT
);

-- ② WORKFLOW GRAPH
CREATE TABLE WorkflowGraph (
  id          TEXT PRIMARY KEY,          -- e.g. 'DocumentVisit'
  label       TEXT NOT NULL,
  description TEXT
);

CREATE TABLE WorkflowNode (
  id          TEXT PRIMARY KEY,
  graph_id    TEXT REFERENCES WorkflowGraph(id),
  label       TEXT NOT NULL,
  is_initial  INTEGER DEFAULT 0,
  is_terminal INTEGER DEFAULT 0,
```

```

    module_hint  TEXT                                -- optional: maps to UI module name
);

CREATE TABLE NodeContextContract (
    node_id      TEXT REFERENCES WorkflowNode(id),
    property_id  TEXT REFERENCES ObjectProperty(id),
    role         TEXT NOT NULL,                      -- 'entry_requires' / 'exit_contributes'
    PRIMARY KEY (node_id, property_id, role)
);

CREATE TABLE WorkflowTransition (
    id            TEXT PRIMARY KEY,
    source_node_id TEXT REFERENCES WorkflowNode(id),
    target_node_id TEXT REFERENCES WorkflowNode(id),
    trigger_command TEXT,                            -- references Command.id
    transition_type TEXT NOT NULL,                    -- 'mandatory'/'optional'/'global'/'conditi
    label         TEXT
);

CREATE TABLE TransitionGuard (
    transition_id TEXT REFERENCES WorkflowTransition(id),
    property_id   TEXT REFERENCES ObjectProperty(id),
    condition     TEXT DEFAULT 'present',             -- 'present'/'not_null'/expression
    PRIMARY KEY (transition_id, property_id)
);

-- ③ COMMAND REGISTRY
CREATE TABLE Command (
    id            TEXT PRIMARY KEY,                  -- e.g. 'cmd.selectPatient'
    label         TEXT NOT NULL,
    rpc_name      TEXT,                              -- Vista RPC, if applicable
    description    TEXT
);

CREATE TABLE CommandContextRequirement (
    command_id   TEXT REFERENCES Command(id),
    property_id  TEXT REFERENCES ObjectProperty(id),
    role         TEXT NOT NULL,                      -- 'requires' / 'produces'
    PRIMARY KEY (command_id, property_id, role)
);

```

This schema creates a **fully navigable, referentially consistent** definition of the entire system model. Any change to an object property name immediately surfaces as a foreign key violation in

the NodeContextContract, TransitionGuard, and CommandContextRequirement tables — preventing silent inconsistencies between the object model and the workflow definition.

SCHEMA MIGRATION

Since the object model is not finalized and will change, the system model database must support **schema versioning**. Add a single metadata table:

Schema Version Tracking

```
CREATE TABLE SchemaVersion (
  version      INTEGER PRIMARY KEY,
  applied_at   TEXT DEFAULT (datetime('now')),
  description  TEXT
);
-- Insert on database creation:
INSERT INTO SchemaVersion (version, description) VALUES (1, 'Initial schema');
```

The authoring tool checks the schema version on startup and applies pending migrations in order. This makes model database upgrades reproducible and auditable — essential when the model is shared across a development team.

ANALYSIS 04

Workflow Graph Editor — Tooling Options

The graph editor is an internal tool — its primary users are developers and clinical informaticists designing workflows, not end users. It does not need to be polished; it needs to be *correct and practical*. There are two viable strategies.

STRATEGY A — EMBED A DELPHI GRAPH COMPONENT

Build the editor as a Delphi application using an existing graph/diagram VCL component for the canvas. The editor saves directly to the SQLite system model database.

[TSimpleGraph \(Khojasteh/simple-graph\)](#)

DELPHI NATIVE VCL

FREWARE + SOURCE

BEST AVAILABLE OPTION

A visual Delphi VCL component for drawing directed graphs. Nodes are customizable geometric shapes (rectangle, ellipse, rhombus for decision nodes, etc.); edges connect nodes with configurable line styles. The component handles all hit-testing, drag-move, and selection. It is mature, source-available, and the most capable free option in the Delphi ecosystem for this purpose.

- **Seven built-in node shapes** — rectangle (state), ellipse (terminal), rhombus (decision/conditional) cover all workflow node types

- **Flexible link objects** — edges can be anchored to nodes; endpoints customizable for arrow heads

- **Hit-testing and selection** — click-to-select nodes and edges; move nodes by drag

- **Clipboard and printer support** — copy/paste graph objects; print workflow diagrams for clinical review

- **Fully customizable** — node appearance (color, label, border) is data-driven; can render node state (initial, terminal, has-errors) visually

- Source code available — can be patched for Delphi 12.2 CE compatibility if needed

Integration approach: Each `TSimpleGraph` node stores the `WorkflowNode.id` in its `Tag/UserData`. On node select, the right panel shows a property editor bound to the SQLite row for that node. Saving writes to the SQLite model database. The graph layout (node x/y positions) is stored in a separate `LayoutData` table — separate from the semantic model.

Caveat on TSimpleGraph: The author has stated no future updates are planned. The component compiles under older Delphi versions; test compatibility with Delphi 12.2 CE before committing. If compilation fails, the source is available to patch. An alternative is to build a custom canvas-based editor using `TCanvas` directly — more work upfront, but zero external dependency and full control.

STRATEGY B — EMBED A WEB-BASED GRAPH EDITOR (RECOMMENDED FOR RICHNESS)

Host a web-based graph editor (running locally, no internet) inside a Delphi form using an embedded Chromium browser component. The web editor communicates with the Delphi host via JavaScript interop to read and write the SQLite model database. This gives you a far richer graph editing experience with no Delphi graph component limitations.

[CEF4Delphi — Chromium Embedded Framework for Delphi](#)

DELPHI NATIVE

MIT / BSD LICENSE

ACTIVELY MAINTAINED 2024

A Delphi/Lazarus wrapper for the Chromium Embedded Framework. Embeds a full Chromium browser in a VCL form. Supports bidirectional JavaScript interop — Delphi can call JavaScript functions and JavaScript can call Delphi-registered handlers. Used to host a local HTML/JS graph editor inside the authoring tool application.

- **Full Chromium engine** — runs any modern JavaScript library including graph editing frameworks
- **JavaScript → Delphi calls** — the web editor calls a Delphi handler to save node/edge data to SQLite
- **Delphi → JavaScript calls** — Delphi loads the graph from SQLite and pushes it to the web editor on startup
- Supports Delphi 12.x; actively maintained by Salvador Díaz Fau

[mxGraph / draw.io Graph Library \(JavaScript\)](#)

APACHE 2.0

EMBEDDED IN CEF4DELPHI

The JavaScript graph library underlying draw.io. Provides a full-featured interactive graph canvas — node creation, edge drawing, drag-move, zoom, undo/redo, custom node shapes and styles, label editing. Runs entirely in the browser; can be hosted locally as static HTML/JS files loaded by CEF4Delphi. This is the same engine used in professional diagramming tools.

- **Professional graph editing** — undo/redo, snap-to-grid, edge routing, zoom/pan, keyboard shortcuts
- **Custom node shapes** — define state, initial, terminal, conditional node styles matching the workflow model
- **Export to SVG/PNG** — workflow diagrams can be exported for clinical review documentation
- **Local deployment** — bundle the JS files with the authoring tool; no internet connection required
- **JavaScript API for programmatic graph manipulation** — load from and save to Delphi via CEF4Delphi interop

Integration approach: mxGraph runs in CEF4Delphi. When the user saves, the JavaScript calls a registered Delphi handler passing the graph as JSON. The Delphi handler parses the JSON and writes nodes, edges, and guards to the SQLite model database. On load, Delphi reads from SQLite and passes the graph JSON to the JavaScript editor via `ExecuteJavaScript`.

Recommendation: If the authoring tool is for internal developer use only, **Strategy A** (**TSimpleGraph**) is faster to implement and has no CEF4Delphi deployment overhead (~150MB Chromium binaries). If the tool will be used by clinical informaticists or domain experts who expect a draw.io-quality experience, **Strategy B (CEF4Delphi + mxGraph)** is significantly more capable and worth the additional setup cost.

ANALYSIS 05

Object Model Schema Editor — Tooling

The object model schema editor is simpler than the workflow editor — it manages a structured table of object types and their properties. No graph canvas is needed. A standard Delphi master-detail form backed by FireDAC and SQLite is entirely sufficient.

FireDAC + SQLite + Standard VCL (Delphi Built-In)

NO EXTERNAL DEPENDENCY

BUNDLED WITH DELPHI 12.2 CE

RECOMMENDED FOR SCHEMA EDITOR

FireDAC is included in Delphi 12.2 CE and provides full SQLite support via static linking — no SQLite DLL required for deployment. A master-detail form with TFDConnection, TFDQuery, TDataSource, TDBGrid, and TDBEdit components provides a complete, functional schema editor with minimal code. Data-aware components handle all binding, navigation, and editing.

- **Static SQLite linking** — add FireDAC.Phys.SQLiteWrapper.Stat to uses clause; no DLL deployment
- **Master-detail** — ObjectType master list → ObjectProperty detail grid; TFDQuery MasterSource/MasterFields wiring
- **Live validation** — TFDQuery with SQL constraints shows referential integrity violations immediately
- **Search and filter** — TFDQuery.Filter or parameterized SQL for finding properties by type
- **Impact analysis query:** "which workflow nodes reference this property?" — a SQL join across NodeContextContract

Key feature to implement: When a property is renamed or deleted, run an impact query to show all workflow nodes and transition guards that reference it before allowing the change. This prevents silent model inconsistency — the single most common failure mode in evolving domain models.

VirtualTreeView

DELPHI NATIVE

MIT LICENSE

ENHANCED NAVIGATION

The most capable tree/list control for Delphi VCL. Significantly more flexible than TTreeView and TListView for rendering the object model hierarchy (object types → properties) with inline editing, icons, multi-column display, and virtual (callback-driven) data loading. Used in RAD Studio's own IDE. Optional — TTreeView suffices for a simple editor, but VirtualTreeView handles large object models without performance degradation.

- Virtual (callback-based) data model — the tree does not store data; it calls back to query what to display
- In-place cell editing — edit property names and types directly in the tree
- Multi-column with custom renderers — property name, type, nullable, description in one row
- Supports Delphi 12.x; actively maintained

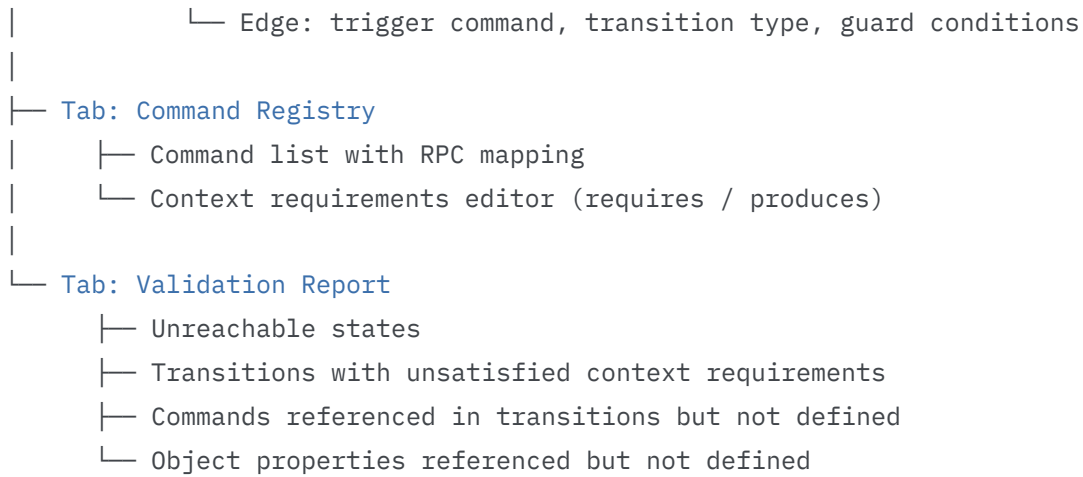
ANALYSIS 06

Recommendation — One Integrated Authoring Tool

Rather than building separate tools for each concern, implement a **single authoring application** — the *System Model Designer* — with tabbed or panel-based views for each layer. This ensures all layers share a single SQLite database connection, making referential integrity checks immediate and cross-layer navigation natural.

System Model Designer – Application Structure

```
SystemModelDesigner.exe
├── Tab: Object Model
│   ├── Left panel: ObjectType list (VirtualTreeView)
│   └── Right panel: ObjectProperty editor for selected type
│       └── Impact query: "used in N workflow nodes, M commands"
├── Tab: Workflow Graph Editor
│   ├── Left panel: WorkflowGraph list
│   ├── Center: Graph canvas (TSimpleGraph or CEF4/mxGraph)
│   └── Right panel: Selected node/edge property editor
│       └── Node: entry_requires, exit_contributes (multiselect from ObjectProp
```



The **Validation tab** is **not optional**. As the model grows, manual consistency checking becomes impossible. The validation report runs a set of SQL queries across all tables and surfaces structural problems: a transition guard referencing a deleted property, a workflow node with no entry_requires despite requiring objects that no prior node contributes, commands used in the graph but absent from the registry. This is the tool that keeps the model coherent as it evolves.

DELIVERABLE SEQUENCE FOR THE AUTHORING TOOL

- **Sprint 1** — SQLite schema creation and migration engine. FireDAC connection. Schema version check on startup. No UI yet — just the database layer working and tested.

- **Sprint 2** — Object Model tab. Master-detail ObjectType → ObjectProperty editor. Rename/delete impact query. This is usable immediately to capture the initial domain object model.

- **Sprint 3** — Command Registry tab. List of commands with RPC mappings and context requirement editor. Validates against the object model.

- **Sprint 4** — Workflow Graph Editor tab. Even with TSimpleGraph as a simple canvas, nodes and edges created and saved to SQLite. Node property panel for context contracts. This is the milestone that enables clinical workflow review.

- **Sprint 5** — Validation Report tab. SQL-based consistency checks across all layers. The model is now self-auditing.

ANALYSIS 07

Supporting External Tools

Several existing external tools complement the authoring workflow without requiring custom development.

DB Browser for SQLite (sqlitebrowser.org)

OPEN SOURCE — GPL

FREE DESKTOP APP

A GUI tool for browsing, querying, and editing SQLite databases. During early development, before the authoring tool's UI is complete, DB Browser provides direct access to the system model database for inspection and manual edits. Also useful for running ad hoc validation queries and for database backup.

- Browse and edit all tables directly — useful before the schema editor UI exists
- Execute arbitrary SQL — run validation queries manually during development
- Import/export CSV — seed the object model from an existing domain dictionary
- No installation required — portable executable

[draw.io](https://app.diagrams.net) (app.diagrams.net) — For Initial Workflow Sketching

FREE / OPEN SOURCE

DESIGN PHASE ONLY

The free web and desktop diagramming tool (formerly draw.io, now diagrams.net). Useful in the early design phase before the authoring tool exists — clinicians and architects can sketch workflow graphs visually and export them as XML. The XML can then be parsed to seed the SQLite workflow graph tables. Not a replacement for the authoring tool, but an excellent starting point.

- No installation required (web version); desktop version available
- Exports to XML — parseable to extract node/edge structure
- Familiar to clinical informaticists; widely used for process documentation
- Use for: initial workflow discovery sessions with clinical stakeholders

Healthcare Application Architecture · Part V

System Model · Priority · Tooling · Delphi 12.2 CE